



**Московский государственный технический
университет им. Н. Э. Баумана
(МГТУ им. Н. Э. Баумана)**

Факультет «Информатика и системы управления»

**Кафедра «Система обработки информации и управления»
(ИУ5)**

Курс «Парадигмы и конструкции языков программирования»

**Лабораторная работа №5-6
«Разработка бота на основе конечного автомата для
Telegram с использованием языка Python»**

Выполнил:
студент группы ИУ5-32Б:
Микаелян С. В.

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.

Задание:

1. Разработайте простого бота для Telegram. Бот должен использовать функциональность создания кнопок.
2. Разработайте бота для Telegram. Бот должен реализовывать конечный автомат из трех состояний.

Код программы:

Файл callback.py

```
from aiogram import Bot, Router, F
from aiogram.types import CallbackQuery

from handlers.func import download_and_send_media
import url_storage as storage

router = Router()

@router.callback_query(lambda callback: 'video' in callback.data or 'audio' in
callback.data)
async def format_selection(callback: CallbackQuery, bot: Bot):
    storage.url_storage = storage.load_url_storage()
    action, url_id = callback.data.split("| ")
    url = storage.url_storage.get(url_id)
    if not url:
        await callback.answer("Ошибка: URL не найден")
        return
    await callback.answer("Загрузка...")

    if action == 'video':
        await callback.message.answer("Начинаю загрузку видео...")
        await download_and_send_media(bot, callback.message.chat.id, url,
media_type = 'video')
    elif action == 'audio':
        await callback.message.answer("Начинаю загрузку аудио...")
        await download_and_send_media(bot, callback.message.chat.id, url,
media_type = 'audio')
```

Файл commands.py

```
from aiogram import Router
from aiogram.filters import CommandStart
from aiogram.types import Message

import handlers.func as hf
import url_storage as storage
import keyboards.inline_kb as in_kb

router = Router()

@router.message(CommandStart())
async def cmd_start(message: Message):
    await message.reply("Привет! Я бот для скачивания видео из ютуба, тиктока
или инстаграмма!\nОтправь мне ссылку и я всё сделаю!")

@router.message(lambda message: "tiktok.com" in message.text or "youtube.com"
in message.text or "youtu.be" in message.text or "instagram.com" in
message.text)
async def video_request(message: Message):
    url = message.text.strip()
    url_id = hf.generate_url_id(url)
    storage.url_storage[url_id] = url
    storage.save_url_storage(storage.url_storage)
    storage.url_storage = storage.load_url_storage()
    await message.answer("Выберите формат загрузки:", reply_markup = await
in_kb.format_button(url_id))
```

Файл func.py

```
import os
import yt_dlp
import time
import hashlib
from aiogram.types import FSInputFile

def generate_url_id(url: str):
    return hashlib.md5(url.encode()).hexdigest()

def get_available_formats(url: str):
    ydl_opts = {
        'quiet': True,
        'listformats': True,
    }

    with yt_dlp.YoutubeDL(ydl_opts) as ydl:
        try:
            info = ydl.extract_info(url, download=False)
            return info.get('formats', [])
        except yt_dlp.DownloadError as e:
            print(f"Ошибка при получении форматов: {e}")
            return []

async def download_and_send_media(bot, chat_id, url, media_type):
    formats = get_available_formats(url)
    if not formats:
        await bot.send_message(chat_id, "Не удалось получить доступные форматы для этого видео.")
        return

    selected_format = None
    if media_type == 'video':
        for f in formats:
            if f.get('vcodec') != 'none' and f.get('acodec') != 'none':
                selected_format = f
                break
    else:
        for f in formats:
            if f.get('ext') == 'm4a':
                selected_format = f
                break

    if not selected_format:
        await bot.send_message(chat_id, f"Не удалось найти подходящий формат для {media_type}.")
        return

    ydl_opts = {
        'format': selected_format['format_id'],
        'outtmpl': f'downloads/{%(title)s}.{"mp4" if media_type == "video" else "m4a"}',
    }

    try:
        start_time = time.time()

        with yt_dlp.YoutubeDL(ydl_opts) as ydl:
            info = ydl.extract_info(url, download=True)
            filename = ydl.prepare_filename(info)

        end_time = time.time()
        elapsed_time = end_time - start_time
```

```

media_file = FSInputFile(filename)
if media_type == "video":
    await bot.send_video(chat_id, media_file, caption=f"Держи. Видео
скачано за {elapsed_time:.2f} секунд")
else:
    await bot.send_audio(chat_id, media_file, caption=f"Держи. Аудио
скачано за {elapsed_time:.2f} секунд")

os.remove(filename)

except Exception as e:
    await bot.send_message(chat_id, f"Произошла ошибка: {e}")

```

Файл inline_kb.py

```

from aiogram.types import InlineKeyboardMarkup, InlineKeyboardButton

async def format_button(url_id):
    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="Видео", callback_data=f"video |
{url_id}")]],
        [InlineKeyboardButton(text="Аудио", callback_data=f"audio |
{url_id}")]],
    ])
    return keyboard

```

Файл bot.py

```

import asyncio
import os

from aiogram import Bot, Dispatcher
from dotenv import load_dotenv

from handlers import callback, commands

async def main():
    load_dotenv()
    token = os.getenv('BOT_TOKEN')
    bot = Bot(token)
    dp = Dispatcher()
    try:
        if not os.path.exists("downloads"):
            os.makedirs("downloads")
        dp.include_router(commands.router)
        dp.include_router(callback.router)
        print('Бот запущен')
        await dp.start_polling(bot)
        await bot.session.close()
    except Exception as ex:
        print(f"{ex}\nБот остановлен")

if __name__ == '__main__':
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print('Бот остановлен')

```

Файл url_storage.py

```
import json
import os

URL_STORAGE_FILE = "url_storage.json"

def load_url_storage():
    if os.path.exists(URL_STORAGE_FILE):
        with open(URL_STORAGE_FILE, "r") as file:
            return json.load(file)
    return {}

def save_url_storage(data):
    with open(URL_STORAGE_FILE, "w") as file:
        json.dump(data, file)

url_storage = load_url_storage()
```

Результат выполнения программы:

